

PaperTrail

Complete project roadmap

Your documents. Your space. A better way to read.

A local-first desktop PDF and EPUB reader built with Vue 3, Vite, TypeScript and Tauri 2. Select a local folder, explore a directory tree and read one document at a time in a focused workspace.

Project	Decision
Repository	HimanshuHD/papertrail-reader
Visibility	Private
Development	GitHub-first; no setup required on your computer
Initial formats	PDF and EPUB
Initial interface	Narrow directory sidebar + large reader pane
First release	Milestones 0-4 plus saved positions and bookmarks
Future scope	Annotations, multiple tabs, split view and more formats

Planning status - 1 October 2026

Repository access has been verified with write permissions. This document is a proposed execution plan, not a report of completed implementation. No delivery dates are committed; estimates should follow the foundation and reader integration spikes.

Contents

1. Product scope and viewer features
2. Architecture and data design
3. Milestones 0-2: foundation and local library
4. Milestones 3-5: readers and productivity
5. Milestones 6-7: reliability and release
6. GitHub-first engineering and CI/CD
7. Initial backlog and release checklist

1. Product scope and viewer features

Core user journey

- Launch the desktop app and choose a folder explicitly.
- Scan that folder recursively for PDF and EPUB documents. Show progress, cancellation and recoverable errors.
- Explore a hierarchical directory tree and open a supported document.
- Read with navigation, progress and display controls; resume the saved position later.
- Refresh the library when files change or choose a different folder.

Priority	Features
P0 - essential	Folder selection, recursive discovery, directory tree, single viewer, loading and error states. PDF page count, page jump, progress slider, zoom, fit width/page and continuous scrolling. EPUB chapters, contents and font sizing.
P1 - everyday reading	Saved positions, bookmarks, document search, outlines/contents, text selection, recent documents, fullscreen, keyboard shortcuts, light/dark themes.
P2 - expansion	Highlights and notes, reading statistics, metadata enrichment, document tabs and session restoration.
P3 - later	Split view, annotation export and additional document formats.

Format-specific behavior

PDF pages are fixed: show current page / total pages and page-based progress. EPUB content reflows: use chapter, location and percentage as the stable navigation model. Any EPUB page count is layout-dependent and may change when fonts or window size change. PDF zoom and EPUB font size are separate controls.

Reader suggestions

Add page rotation for scanned PDFs, optional thumbnails, a focus mode that hides the sidebar, adjustable EPUB line spacing and margins, and a shortcut help panel after the core reading path is stable. Image-only PDFs require OCR for text search; OCR is outside the first release. DRM-protected EPUB support is outside the initial scope.

2. Architecture and data design

Layer	Technology / responsibility
UI	Vue 3 + TypeScript; accessible components and split layout
Build and styling	Vite; Tailwind CSS with shared design tokens
State and routing	Pinia for library, reader and preferences; Vue Router for app views
PDF adapter	PDF.js; worker-backed parsing and lazy page rendering
EPUB adapter	epub.js; reflow, chapters, contents and CFI positions
Desktop boundary	Tauri 2 + Rust; explicit folder selection and scoped filesystem commands
Persistence	Versioned local preferences and reading-state store; SQLite if indexing scale warrants it
Validation	Vitest + Vue Test Utils, Rust tests and Playwright web-shell tests

Dependency flow

UI components call Pinia actions and format-independent reader services. PDF and EPUB adapters manage rendering and report navigation state. A library service calls scoped Tauri commands for indexing and reads. Native filesystem operations remain outside Vue components.

Domain models

- **LibraryRoot**: selected canonical folder, scan status and indexed entries.
- **Document**: stable ID, format, relative path, size, modified time and optional metadata.
- **ReaderSession**: active document and format-specific current position.
- **ReadingState**: PDF page or EPUB CFI, progress, bookmarks and update time.
- **ViewerAdapter**: open, close, navigate, read progress, contents and supported capabilities.

Identity and persistence

Use a generated document ID mapped to file metadata. Add lazy fingerprints to reconcile moves and renames; avoid hashing the whole library during each scan. Treat ambiguous matches explicitly. Store EPUB positions as CFI and PDF positions by page. Version storage schemas and test migrations.

Folder organization

src/components/{layout,library,viewer}; src/features/{library,pdf,epub,bookmarks};
src/{composables,router,stores,types,views,assets}; src-tauri/{src,capabilities,icons}; tests/{unit,e2e};
docs/{architecture,roadmap,development}; .github/{workflows,ISSUE_TEMPLATE}. Keep format-specific code inside its feature.

Local document safety

Restrict native reads to user-approved roots; canonicalize paths and prevent symlink escapes. Handle EPUB content as untrusted: disable document scripting, restrict external resource loading and bound archive extraction. Release capability and CSP settings must be reviewed before broad testing.

3. Milestones 0-2

M0 - Repository and engineering foundation

Deliverable: A buildable, testable desktop shell.

- Inspect main and create chore/project-bootstrap.
- Initialize Vue, Vite and TypeScript with a committed lockfile.
- Add Tauri 2, Tailwind, Pinia and Router.
- Configure lint, formatting, type checking, Vitest and Rust checks.
- Add README, architecture, roadmap, issue and PR templates, and CI.
- Configure branch rules where supported by the account plan.

Exit criteria: Clean CI install, lint, type check, representative tests and frontend build pass. Native compile verification passes on the initial target platform. Bootstrap PR records results.

M1 - UI foundation and application shell

Deliverable: A usable directory-and-reader workspace.

- Build resizable/collapsible sidebar, main reader, toolbar and status bar.
- Use labeled sample data; add empty, loading and error states.
- Implement theme preferences, focus behavior and keyboard access.
- Define viewer controls by adapter capabilities.

Exit criteria: Layout remains usable at small desktop window sizes. Sidebar and toolbar are keyboard accessible. Themes persist, and mock content is clearly identified.

M2 - Local filesystem integration

Deliverable: A real local document library.

- Implement explicit folder dialog and recursive Rust scanner.
- Match PDF/EPUB extensions case-insensitively; verify format when opened.
- Scan incrementally with cancellation and bounded concurrency.
- Render a hierarchical tree; report inaccessible paths without failing all results.
- Persist approved root selection and add refresh/rescan.

Exit criteria: Only the approved root is scanned. Nested files appear correctly. Cancellation, unreadable directories, empty folders, symlinks and mixed-case extensions are tested.

4. Milestones 3-5

M3 - PDF reader

- Integrate PDF.js worker and release resources when a document closes.
- Render visible pages lazily; cancel obsolete rendering during zoom/navigation.
- Add current/total pages, page jump, progress slider and continuous scroll.
- Add zoom, fit width/page, text selection, outlines and basic search.
- Handle password requests, invalid PDFs and unavailable files.

Exit criteria: Page navigation and slider stay synchronized. Zoom preserves a useful reading position. Search works for text PDFs; scanned documents show the limitation. Large-document memory use is measured.

M4 - EPUB reader

- Integrate epub.js behind the shared adapter contract.
- Add chapters, contents, location-based progress and navigation.
- Add font sizing, reading themes, margins and line spacing.
- Generate locations without blocking UI and persist/restore CFI.
- Block EPUB scripts and handle malformed archives or unsupported content.

Exit criteria: A reopened EPUB resumes its position across font and window changes. Contents navigation works. Reflow does not imply a fixed total page count.

M5 - Reading productivity

- Ship saved positions and bookmarks for the first release.
- Add recent documents, library filtering and metadata display.
- Refine fullscreen, shortcuts, focus handling and accessibility.
- After the first release, add highlights/notes and reading statistics.
- Define annotation selectors and local schema migrations before storing notes.

Exit criteria: Positions and bookmarks survive restart. Missing files produce a recoverable state. Reader and library search are clearly distinguished. Annotation work has format-specific persistence tests.

First-release boundary

Release 0.1 should complete M0-M4 and the saved-position/bookmark subset of M5. Search, accessibility and resource management remain part of the release quality gate. Advanced annotations and statistics can follow without delaying a useful reader.

5. Milestones 6-7

M6 - Performance and reliability

- Profile scan throughput, time to first page, scrolling, zoom and memory on representative devices.
- Use lazy/virtual rendering for PDFs and tree virtualization when library scale requires it.
- Verify resource cleanup across repeated document switching.
- Reconcile moved, renamed and deleted documents with clear recovery choices.
- Test native path authorization, malformed documents and bounded EPUB decompression.
- Add opt-in, local-first diagnostics that avoid document contents and private paths.

Exit criteria: No sustained memory growth during switching; scanning remains cancellable and the UI responsive. Record measurements and agreed budgets against a named test machine and dataset before calling performance complete.

M7 - Multi-document experience and release

- Introduce tabs with isolated viewer sessions and explicit inactive-tab resource limits.
- Restore the last session with handling for unavailable files.
- Add optional split view after tab behavior is stable.
- Build platform installers through GitHub Actions. Start with Windows as the proposed primary target, then validate macOS and Linux.
- Configure signing and notarization when credentials are available. Do not describe unsigned builds as signed.
- Publish versioned release notes, known limitations, checksums and user documentation.

Exit criteria: Tab state is isolated, restart restoration works and installer smoke tests pass on supported operating systems. Release artifacts are traceable to a reviewed commit.

Testing matrix

Area	Representative cases
Library	Empty/nested folders, thousands of entries, Unicode paths, permission denial, symlinks and cancellation
PDF	Text, image-only, password-protected, malformed and large multi-page documents
EPUB	Reflowable books, complex contents, missing metadata, malformed archives and blocked remote resources
Persistence	Restart, migration, moved/deleted file, font change and window resize
Desktop	Folder dialog, scoped native reads, install/launch and platform-specific rendering
Accessibility	Keyboard-only reading, focus visibility, labels, contrast and screen-reader checks

6. GitHub-first engineering and CI/CD

Development workflow

Issue → branch from main → focused commits → pull request → automated checks → review → squash merge → delete branch. Main stays releasable. Use feature/*, fix/* and chore/* branches. Link issues in PRs; use closing keywords only for work actually completed.

All source changes are delivered through GitHub commits and PRs. No developer setup is required on your computer. GitHub Actions runners install and build the project. Real desktop testing will still be needed for dialogs, permissions and installers.

Workflow	Triggers and purpose
CI	PRs to main and pushes to main: npm ci, lint, format check, type check, unit tests and frontend build; Rust fmt, clippy and tests when applicable.
Desktop verification	Native/config changes, manual runs and release preparation: platform prerequisites, Rust compile and installer artifacts. Broaden to supported OS matrix.
E2E	PRs for core flows: Playwright web-shell tests with controlled fixtures. Native integration needs platform-capable tests and desktop smoke tests.
Dependency review	Dependency PRs and scheduled checks: review advisories and lockfile changes; triage actual impact.
Release	Reviewed version tags: build platform packages, attach checksums and release notes. Use drafts for pre-release review.

Pipeline rules

- Commit lockfiles and make scripts match the workflow commands. Pin supported versions after verification; do not copy stale example workflows blindly.
- Give CI read-only repository permissions. Limit write permissions to release jobs.
- Pin third-party actions to immutable commit SHAs and maintain those pins.
- Use concurrency cancellation for obsolete PR checks; avoid duplicate branch and PR builds.
- Keep secrets out of code and PR logs. Restrict signing credentials to protected release environments.
- Retain useful build artifacts and failure logs. Require stable named checks before merge where repository rules are available.

Review checklist

PR purpose and linked issue; user-visible behavior; validation evidence; screenshots for UI changes; native capability changes; migration implications; known limitations and recovery. Avoid merging failing checks.

7. Initial backlog and release checklist

The IDs below are planning labels, not existing GitHub issue numbers. Create actual issues after inspecting the repository and checking for duplicates.

ID	Task	Milestone
B01	Vue + Vite + TypeScript foundation	M0
B02	Tailwind and design tokens	M0-M1
B03	Tauri 2 desktop shell	M0
B04	Pinia, Router and service boundaries	M0-M1
B05	Main reader layout and states	M1
B06	Themes and keyboard accessibility	M1
B07	Native folder selection and scoped access	M2
B08	Recursive scanner and cancellation	M2
B09	Directory tree and rescan	M2
B10	PDF.js integration and text layer	M3
B11	PDF navigation, progress and zoom	M3
B12	Stable identity and saved reading positions	M2-M5
B13	EPUB adapter and CFI persistence	M4
B14	Search, contents and bookmarks	M3-M5
B15	GitHub Actions checks and PR templates	M0
B16	Desktop validation and release automation	M0-M7

Release 0.1 acceptance checklist

- Approved folders yield a correct PDF/EPUB tree and can be rescanned.
- Both formats open and navigate with appropriate progress controls.
- Saved positions and bookmarks restore correctly.
- Corrupt, locked, inaccessible and missing files have useful error states.
- No cloud upload of document contents is required.
- Automated checks pass, and a desktop build is smoke-tested on the declared supported OS.
- README explains installation, supported formats, limitations and troubleshooting.

Immediate next implementation step

Inspect the current main branch, create chore/project-bootstrap, add the foundation and CI, then open a reviewable PR. Keep filesystem and real document rendering behind later milestones; do not present sample UI as working reader functionality.